

4-bit Unsigned Array Multiplier

From RTL to GDSII with Open-Source Tools

Digital VLSI Design — Team Assignment Report

SkyWater SKY130 (130 nm) · `sky130_fd_sc_hd` standard cells

Team member	Roll number
Anumay Rai	2025122013
Ashlin K Sunil	2024102025

Repository: `github.com/AnumayRai/RTL-to-GDS`

25 October 2026

Contents

1 Design Overview **2**

2 Part A — RTL Design and Verification **2**

2.1 Array multiplier architecture . . . 2
2.2 Registers and latency . . . 2
2.3 Block diagram . . . 3
2.4 Testbench and simulation results . . . 3
2.5 Hand-worked example: $A = 8$, $B = 7$ . . . 3
2.6 Waveform and two-cycle latency . . . 4

3 Part B — Logic Synthesis with Yosys **6**

4 Part C — Static Timing Analysis with OpenSTA **7**

5 Part D — Physical Design: RTL to GDSII **8**

5.1 Configuration and flow . . . 8
5.2 Clock-period selection and run record . . . 8
5.3 Results . . . 9
5.4 Layout . . . 10

6 Answers to the Report Questions **12**

7 Version History and Deliverables **14**

Summary of results. The multiplier passes exhaustive simulation of all 256 input pairs at RTL and gate level (the fault-injection test is caught with 95 errors). Yosys maps it to 65 SKY130 cells (16 flip-flops, 751.97 µm^2). OpenSTA gives $T_{\min} = 2.676$ ns and $F_{\max} = 373.69$ MHz at the typical corner. LibreLane produces a GDSII layout on a 150 µm × 150 µm die that meets setup timing at all corners with `CLOCK_PERIOD` = 7.5 ns (worst setup slack +0.456 ns), with 0 DRC errors and 0 LVS errors.

1 Design Overview

The design is a 4-bit $\times$ 4-bit unsigned array multiplier with an 8-bit product. It is written in structural Verilog (twelve directly instantiated full adders, no $*$ operator), registers both operands and the product, and has an active-low asynchronous reset. Table 1 lists the ports of the top module `mult_array`.

Table 1: Ports of `mult_array`.

Port	Direction	Width	Description
`clk`	input	1	Clock, rising edge
`rst_n`	input	1	Asynchronous reset, active low: clears all registers to 0
`a`	input	4	Multiplicand
`b`	input	4	Multiplier
`p`	output	8	Product $a \times b$, registered

The flow used is shown below; each stage is described in the corresponding part of this report.

Part A RTL + simulation (Icarus, GTKWave) → **Part B** Synthesis + GLS (Yosys, Icarus) → **Part C** Static timing (OpenSTA) → **Part D** Place & route, GDSII (LibreLane)

2 Part A — RTL Design and Verification

2.1 Array multiplier architecture

Multiplication is shift-and-add. For every bit b_i of B there is one partial product $pp_i = A \wedge b_i$ (each bit of A ANDed with b_i), shifted left by i places; adding the four partial products gives the product. In hardware, with $pp_i[j] = a_{\text{reg}}[j] \wedge b_{\text{reg}}[i]$:

- **Row 0** is only four AND gates. $p[0] = pp_0[0]$; the other bits pass down as $r_0 = \{0,\ pp_0[3],\ pp_0[2],\ pp_0[1]\}$.
- **Rows 1–3** are each a 4-bit ripple-carry adder of four full adders. Row i adds pp_i to r_{i-1} with carry-in 0. The lowest sum bit is a finished product bit ($p[1]$, $p[2]$, $p[3]$); the other three sums and the final carry pass down as $r_i = \{c_i,\ s_i[3],\ s_i[2],\ s_i[1]\}$.
- **Row 3** also produces the top bits: $p[7{:}4] = \{c_3,\ s_3[3],\ s_3[2],\ s_3[1]\}$.

In total the array uses $N^2 = 16$ AND gates and $N(N-1) = 12$ full adders. Each full adder is the module `full_adder`, written with `assign` statements:

$$\text{sum} = a \oplus b \oplus c_{\text{in}}, \qquad c_{\text{out}} = ab + a\,c_{\text{in}} + b\,c_{\text{in}}.$$

2.2 Registers and latency

On every rising clock edge `a` and `b` are captured into the 4-bit registers `a_reg` and `b_reg`; the array computes `prod_comb` from the registered values, and the next rising edge captures it into the 8-bit register `p`. All registers use `always @(posedge clk or negedge rst_n)` and clear to 0 when `rst_n` = 0. This gives exactly $4 + 4 + 8 = 16$ flip-flops and a latency of 2 clock cycles (throughput one result per cycle).

2.3 Block diagram

Figure 3 shows the complete hardware: the two input registers, the 16 AND gates (labelled `pp`ij $= a_j \wedge b_i$), the twelve full adders FA0–FA11 with their ripple carries (`cc1`–`cc9`) and row carries (`c1`–`c3`), and the 8-bit output register.

Because of its size, the diagram is shown on its own landscape page at the end of Part A (page 5).

2.4 Testbench and simulation results

`tb/tb_mult.v` generates a 10 ns clock, holds `rst_n` low for the first two cycles, applies all 256 combinations ($a = 0 \ldots 15$, $b = 0 \ldots 15$) on the falling edge, waits two rising edges for the latency, and compares `p` with $a * b$ (the $*$ operator is used only in the testbench). It prints every mismatch, one summary line, and writes `mult.vcd`.

```
iverilog -g2012 -o sim tb/tb_mult.v rtl/full_adder.v rtl/mult_array.v
vvp sim
PASS: 256 vectors checked, 0 errors
```

Error-detection test. With `sum` in `full_adder.v` temporarily changed to `a ^ b`, the same testbench reports:

```
FAIL: 256 vectors checked, 95 errors
```

The change was then undone and the design passed again, confirming that the testbench really catches wrong hardware.

2.5 Hand-worked example: $A = 8$, $B = 7$

The example uses the roll number of **Anumay Rai (2025122013)**: last digit 3, so $A = 3 + 5 = 8$ and $B = 7$. In the layout of Section 2 of the assignment:

1 0 0 0	A = 8
× 0 1 1 1	B = 7
1 0 0 0	pp0 ($b_0 = 1$)
1 0 0 0	pp1 ($b_1 = 1$), shift 1
1 0 0 0	pp2 ($b_2 = 1$), shift 2
0 0 0 0	pp3 ($b_3 = 0$), shift 3
0 0 1 1 1 0 0 0	P = 56

Row	Adds	Sum	Carry	Out
0	—	—	—	$p[0] = 0$
1	$1000 + 0100$	1100	0	$p[1] = 0$
2	$1000 + 0110$	1110	0	$p[2] = 0$
3	$0000 + 0111$	0111	0	$p[3] = 1$
$p[7{:}4] = \{c_3, s_3[3{:}1]\} = 0011$				
$P = 00111000_2 = 32 + 16 + 8 = 56$				

The right-hand table follows the hardware rows ($r_0 = 0100$, $r_1 = 0110$, $r_2 = 0111$). The team's handwritten working, which follows the same rows, is reproduced in Figure 1.

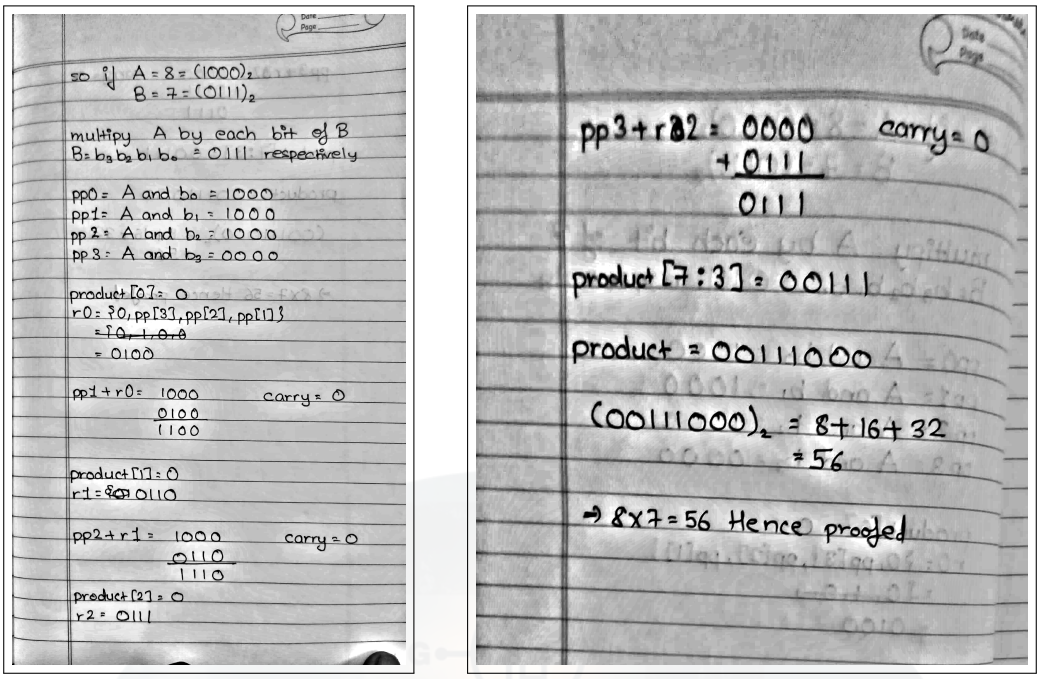


Figure 1: Handwritten working of 8×7 through the three adder rows of the array (`report/8-7Multiply.pdf`; contrast enhanced for printing).

2.6 Waveform and two-cycle latency

In the simulation the pair $a = 8$, $b = 7$ is applied at the falling edge at 2730 ns. At rising edge 1 (2735 ns) the operands are captured into `a_reg`/`b_reg`, while `p` still shows the previous result 48 (8×6). At rising edge 2 (2745 ns) the product register captures the result and `p` becomes **56**. The latency is therefore two clock cycles (2×10 ns in the testbench).

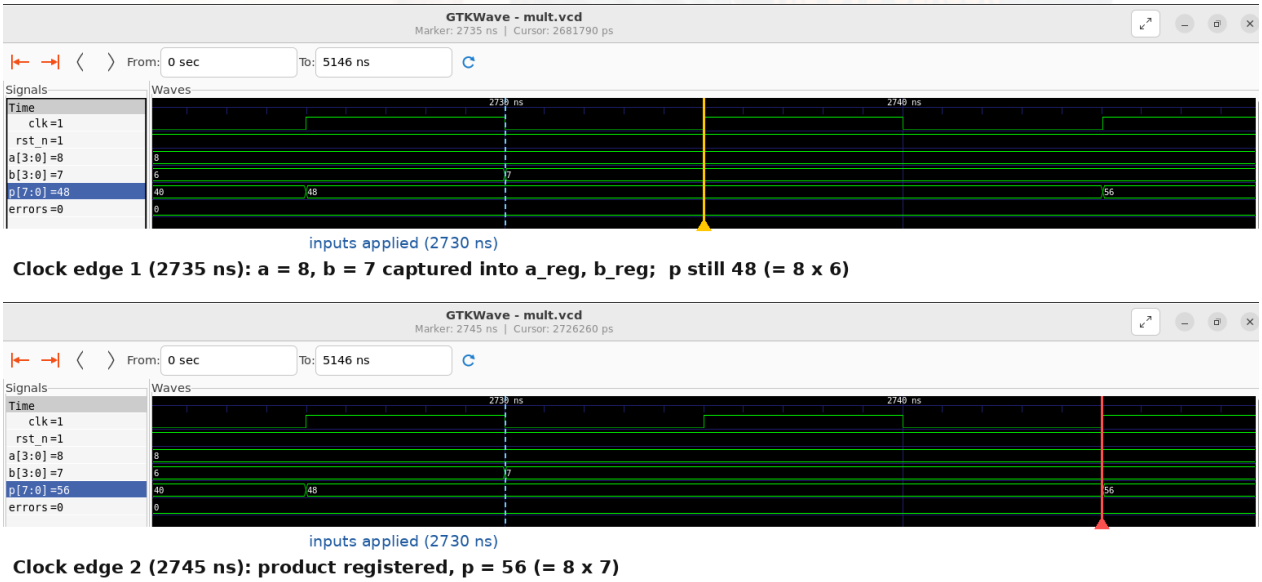


Figure 2: GTKWave view (decimal format) of `clk`, `rst_n`, `a`, `b` and `p` around the 8×7 vector. Top: marker at edge 1; bottom: marker at edge 2. Annotations added; screenshots in `report/multiplier_waveform_8x7_cursor1/2.png`.

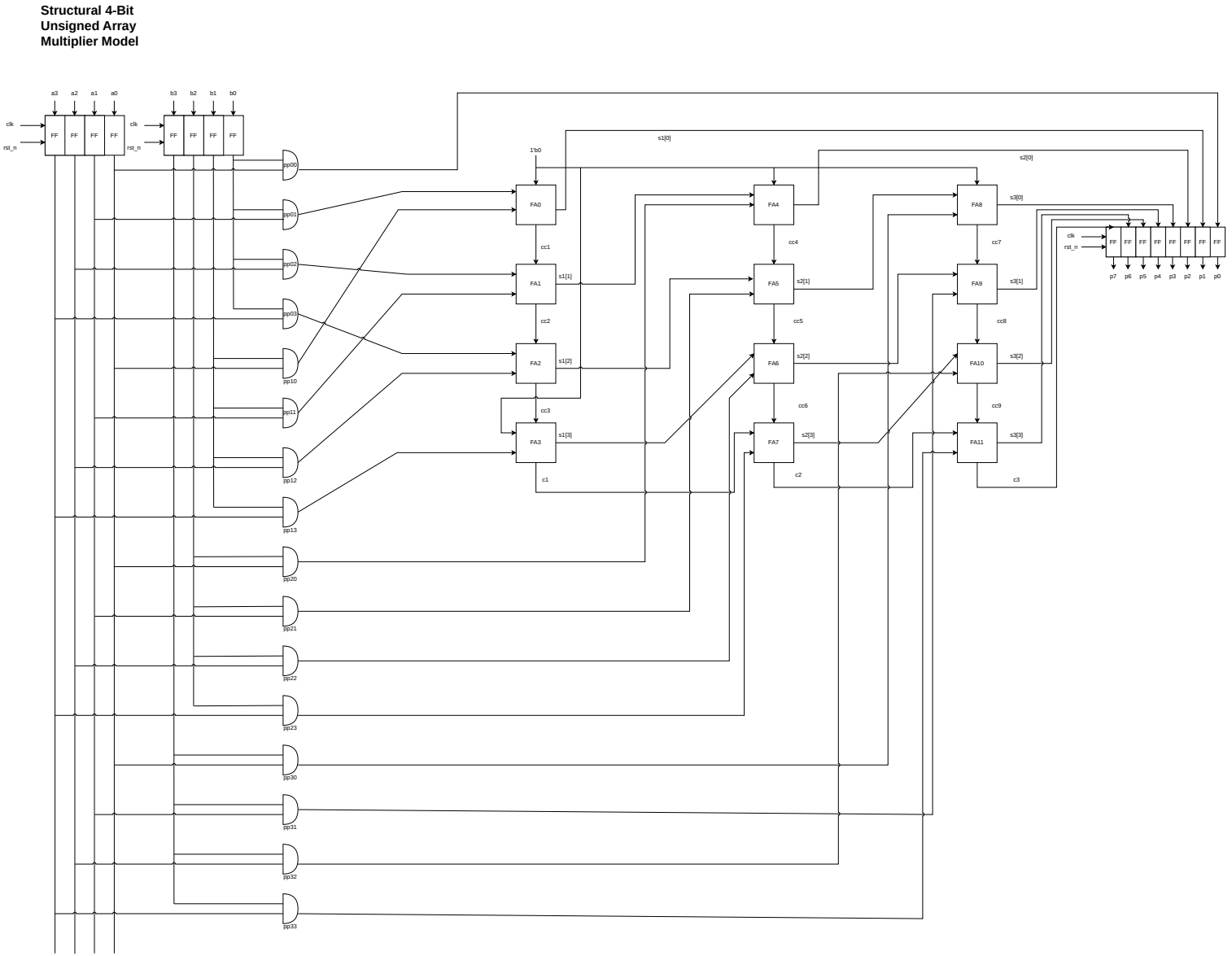


Figure 3: Block diagram of `mult_array` (drawn by the team in draw.io; every connection checked against `rtl/mult_array.v`).

3 Part B — Logic Synthesis with Yosys

`synth/synth.tcl` performs the seven required steps and is run with `yosys -c synth/synth.tcl`:

```
yosys read_verilog rtl/full_adder.v rtl/mult_array.v
yosys synth -top mult_array -flatten
yosys dfflibmap -liberty $lib
yosys abc -liberty $lib
yosys opt_clean -purge
yosys tee -o synth/mult_array_stat.rpt stat -liberty $lib
yosys write_verilog -noattr synth/mult_array_netlist.v
```

Table 2: Part B results table.

Metric	Value
Total number of cells	65
Number of flip-flops	16 (`sky130_fd_sc_hd__dfrtp_1`)
Total cell area (μm^2)	751.971
Flip-flop area (μm^2)	400.384 (16×25.024)
Flip-flop area (% of total)	53.24 %
The 3 most-used cell types and their counts	`dfrtp_1`: 16; `nand2_1`: 7; then a four-way tie at 5: `a21oi_1`, `nor2_1`, `xnor2_1`, `xor2_1`
Netlist simulation result (PASS/FAIL line)	`PASS: 256 vectors checked, 0 errors`

Gate-level simulation. The mapped netlist was simulated with the same testbench and the SKY130 cell models (`primitives.v`, `sky130_fd_sc_hd.v`); all 256 vectors passed, so synthesis did not change the behaviour.

4 Part C — Static Timing Analysis with OpenSTA

`sta/sta.tcl` reads the typical-corner library `sky130_fd_sc_hd__tt_025C_1v80.lib`, reads and links the netlist, creates the clock on `clk`, sets 1.0 ns input and output delays, reports the worst setup slack and writes the critical-path report. It was run twice: first with $T = 10$ ns, then with $T = T_{\min}$ rounded to 0.01 ns.

```
sta -exit sta/sta.tcl # set T 10 -> worst slack max 7.32 (report: 7.324)
sta -exit sta/sta.tcl # set T 2.68 -> worst slack max 0.00 (report: 0.004)
```

$$T_{\min} = T - \text{WNS} = 10 - 7.324 = 2.676 \text{ ns}, \qquad F_{\max} = \frac{1000}{T_{\min}} = \frac{1000}{2.676} = 373.69 \text{ MHz}.$$

Table 3: Part C results table.

Metric	Value
Worst slack at $T = 10$ ns (ns)	7.324
$T_{\min}$ (ns)	2.676 (2.68 rounded)
$F_{\max}$ (MHz)	373.69
Slack at $T = T_{\min}$ (ns)	+0.004 (at $T = 2.68$ ns)
Critical path start point	`_102_` = `b_reg[0]` (`dfrtp_1`, CLK)
Critical path end point	`_096_` = `p[6]` (`dfrtp_1`, D)
Number of cells on the critical path	10 (8 logic cells + launch and capture flip-flops)
Latency (ns) $= 2 \times T_{\min}$	5.352
Throughput (million results per second)	373.69

Table 4: Cells on the critical path (from `sta/mult_array_timing.rpt`, $T = 2.68$ ns).

#	Instance	Cell	Delay (ns)	Arrival (ns)	Role
1	`_102_`	`dfrtp_1`	0.386	0.386	launch flip-flop (`b_reg[0]`), CLK→Q
2	`_042_`	`and4_1`	0.251	0.638	partial products
3	`_048_`	`nor3_1`	0.217	0.855	row-1 adder logic (merged)
4	`_050_`	`o21ai_0`	0.173	1.028	row-1 adder logic (merged)
5	`_061_`	`maj3_1`	0.404	1.431	full-adder carry
6	`_072_`	`xnor2_1`	0.166	1.598	full-adder sum
7	`_082_`	`maj3_1`	0.389	1.987	full-adder carry
8	`_087_`	`maj3_1`	0.380	2.367	full-adder carry
9	`_089_`	`xnor3_1`	0.175	2.542	full-adder sum (`FA11.sum`)
10	`_096_`	`dfrtp_1`	setup 0.134	req. 2.546	capture flip-flop (`p[6]`)

Full-adder count on the path. Five of the eight logic cells are clearly full-adder cells (three `maj3` carries and the `xnor2`/`xnor3` sums); `and4`, `nor3` and `o21ai` combine partial-product ANDs with first-row adder logic. The eight logic stages are of the same order as the estimate of about $2N = 8$ full-adder delays from Section 2 of the assignment. The asynchronous-reset recovery check (`rst_n`) has much larger slack (9.228 ns at 10 ns, 1.908 ns at 2.68 ns) and is not critical.

5 Part D — Physical Design: RTL to GDSII

5.1 Configuration and flow

LibreLane runs synthesis, floorplan (fixed 150 × 150 µm die and power grid), placement, clock-tree synthesis, routing, parasitic extraction, signoff STA, DRC (Magic, KLayout) and LVS (Netgen), and writes the GDSII. `pnr/config.json`:

```
{ "DESIGN_NAME": "mult_array",
  "VERILOG_FILES": ["dir::../rtl/full_adder.v", "dir::../rtl/mult_array.v"],
  "CLOCK_PORT": "clk", "CLOCK_PERIOD": 7.5,
  "FP_SIZING": "absolute", "DIE_AREA": [0, 0, 150, 150] }
```

```
cd pnr
librelane config.json
cp runs/VERIFY_7P5_REPEAT/final/metrics.json metrics.json
cp runs/VERIFY_7P5_REPEAT/final/gds/mult_array.gds mult_array.gds
klayout mult_array.gds
```

5.2 Clock-period selection and run record

The starting period follows the assignment rule:

$$\texttt{CLOCK_PERIOD} = 1.2 \times T_{\min} = 1.2 \times 2.676 = 3.211 \text{ ns} \;\rightarrow\; 3.5 \text{ ns (next 0.5 ns)}.$$

Whenever setup timing failed, `CLOCK_PERIOD` was increased by 0.5 ns and the flow was run again. LibreLane checks setup timing at nine process/voltage/temperature corners; the reported worst setup slack is the minimum over all of them and is set by the slow corner `ss_100C_1v60` (1.60 V, 100 °C), where cells are roughly 1.9× slower than at the typical corner. Table 5 lists every run.

Table 5: All 13 LibreLane runs (values from each run's `final/metrics.json`). Slack values are the worst over all corners.

Run	Period	Setup WS (ns)	TNS (ns)	Viol.	DRC	LVS	Timing
RUN_2026-10-24_20-38-54	3.5 ns	−2.093688	−8.112946	15	0	0	FAIL
RUN_2026-10-25_01-53-19	4.0 ns	−1.593688	−5.612946	15	0	0	FAIL
RUN_2026-10-25_02-04-14	4.5 ns	−1.093721	−3.188081	12	0	0	FAIL
RUN_2026-10-25_02-40-47	5.0 ns	−0.595666	−1.527877	9	0	0	FAIL
RUN_2026-10-25_02-45-27	5.5 ns	−0.106871	−0.121449	5	0	0	FAIL
RUN_2026-10-25_02-28-42	6.0 ns	−0.040062	−0.040062	3	0	0	FAIL
RUN_2026-10-25_02-50-24	6.5 ns	−0.155348	−0.241024	6	0	0	FAIL
RUN_2026-10-25_04-15-04	6.5 ns	−0.155348	−0.241024	6	0	0	FAIL
RUN_2026-10-25_04-17-52	7.0 ns	−0.043609	−0.043609	2	0	0	FAIL
RUN_2026-10-25_04-22-50	7.5 ns	+0.456391	0	0	0	0	PASS
VERIFY_7P5_REPEAT	**7.5 ns**	**+0.456391**	**0**	**0**	**0**	**0**	PASS
VERIFY_8P0	8.0 ns	+0.956391	0	0	0	0	PASS
VERIFY_8P5	8.5 ns	+1.456391	0	0	0	0	PASS

7.5 ns is the shortest passing period. Every run had 0 DRC (Magic, KLayout and routing) and 0 LVS errors; only setup timing separated them. The 6.0 ns run was closer to passing than the 6.5 ns runs: placement, buffering and routing change between runs, so timing does not improve strictly monotonically with the period, and the measured values are kept as recorded. The submitted `metrics.json` and `mult_array.gds` come from the bold run.

5.3 Results

Table 6: Part D results table (final run `VERIFY_7P5_REPEAT`).

Metric	Value
CLOCK_PERIOD used (ns)	7.5
Number of runs needed	13 (first passing design on run 10; runs 11–13 were confirmation runs)
Core area (μm^2)	17 759.5 (die area 22 500)
Cells after synthesis (from Part B)	65
Cells after place and route	4 831 (339 excluding fill/decap)
Kinds of cells added	fill/decap (4 492), tap (245), timing-repair buffers (20), clock buffers (3)
Wirelength (µm)	2 500
Setup slack (ns)	+0.456 (worst corner)
Post-route F_{max} (MHz)	141.97
F_{max} from Part C (MHz)	373.69
DRC errors	0
LVS errors	0

$$F_{\text{max,post-route}} = \frac{1000}{\texttt{CLOCK_PERIOD} - \text{setup slack}} = \frac{1000}{7.5 - 0.456} = 141.97\ \text{MHz}.$$

This uses the slack reported in `metrics.json`, which comes from the slow corner. For a like-for-like comparison with Part C (typical corner), the routed design's typical-corner setup slack is +3.817 ns, giving $1000/(7.5 - 3.817) = 271.6$ MHz — still below the 373.69 MHz of Part C (see Question 4).

Cells after place and route. The 339 standard cells are 16 flip-flops, 55 multi-input logic cells (LibreLane re-synthesises the RTL with its own settings), 20 timing-repair buffers, 3 clock buffers and 245 tap cells; the remaining 4 492 instances are fill and decap cells that occupy the empty row space.

5.4 Layout

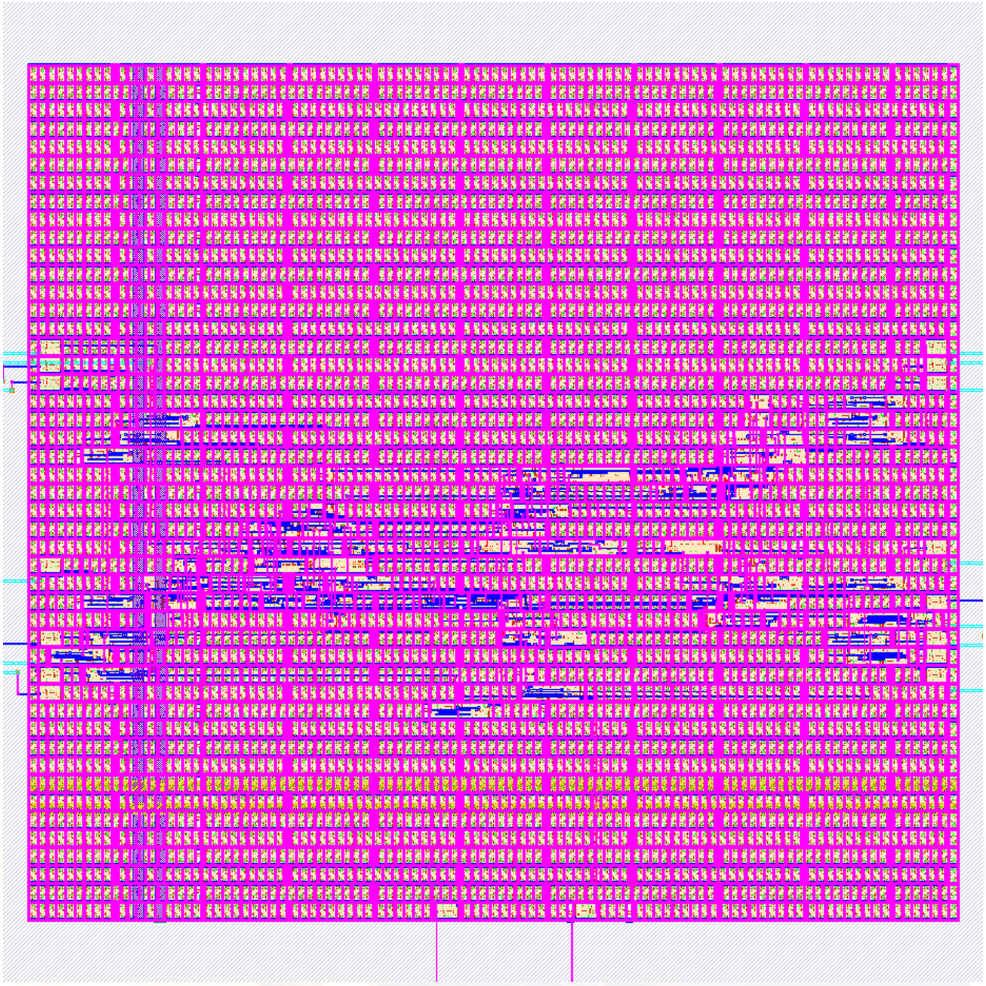


Figure 4: Whole die (150 µm × 150 µm) of the final GDSII in KLayout. The logic occupies the centre band; the remaining rows are filled with decap and fill cells.



Figure 5: Zoomed KLayout view of `pnr/mult_array.gds`, showing standard-cell rows and the metal routing between cells.

6 Answers to the Report Questions

Q1. Logic cells, and why not simply 16 + 12

Synthesis produced 65 cells, of which 16 are flip-flops (`dfrtp_1`), so **49 cells implement the logic**. This is not $16 + 12 = 28$ because a full adder is not one library cell: its sum needs XOR/XNOR cells (`xor2`, `xnor2`, `xnor3`) and its carry a majority function (`maj3`, or combinations such as `a21oi`/`o21ai`). ABC also restructures the logic, merging partial-product AND gates with adder logic into compound cells (`and4`, `a22oi`, `a211oi`, `nor3`) and building other functions from several simple gates (`nand2`, `nor2`). The count therefore reflects how the logic was mapped to the SKY130 library, not the number of RTL modules.

Q2. Critical path

At $T = 2.68$ ns the critical path runs from input register **`b_reg[0]`** (`_102_`) to output register **`p[6]`** (`_096_`, D input `FA11.sum`). It contains **10 cells: 2 flip-flops and 8 logic cells** (`and4` → `nor3` → `o21ai` → `maj3` → `xnor2` → `maj3` → `maj3` → `xnor3`). Five of the eight are clearly full-adder cells (three `maj3` carries, the `xnor2`/`xnor3` sums); the other three combine the partial-product ANDs with first-row adder logic. The estimate of about $2N = 8$ full-adder delays has the same order as the 8 logic stages found. The logic delay is 2.156 ns (about 0.27 ns per cell), plus 0.386 ns clock-to-Q and 0.134 ns setup time.

Q3. Flip-flop share of the cell area

Flip-flops take 400.384 of 751.971 µm^2, i.e. **53.24 %**. Each `dfrtp_1` is 25.02 µm^2, about 3.5 times the average logic cell ($351.59/49 = 7.18$ µm^2). The design is so small that its 49 logic cells do not outweigh the 16 flip-flops it always needs ($4 + 4 + 8$). Registers grow roughly as $4N$ while the array logic grows as N^2, so this share falls for wider multipliers. After place and route the share is 27.47 % (420.403/1530.22 µm^2) because buffers and tap cells add area.

Q4. Why post-route F_{max} is lower, and which cells were added

Post-route $F_{max} = 1000/(7.5 - 0.456) =$ **141.97 MHz** versus **373.69 MHz** from Part C. Reasons: (1) the slack in `metrics.json` comes from the worst corner `ss_100C_1v60`, where cells are about 1.9× slower, while Part C used only the typical corner — even at the typical corner the routed design gives $1000/(7.5 - 3.817) =$ **271.6 MHz**, still lower; (2) the routed wires add resistance and capacitance; (3) the clock goes through a real clock tree with insertion delay and 0.27 ns skew instead of an ideal clock; (4) LibreLane adds a 0.25 ns clock-uncertainty margin. The instance count rises from 65 to **4 831** (339 excluding fill/decap): **4 492 fill and decap cells** (fill cells close gaps in the rows so wells and power rails stay continuous; decap cells store charge to steady the supply); **245 tap cells** (tie the n-well to VPWR and the substrate to VGND to prevent latch-up); **20 timing-repair buffers** (fix slew and capacitance and drive longer nets); **3 clock buffers** (the clock tree that delivers `clk` to all 16 flip-flops with low skew).

Q5. DRC and LVS

DRC (design rule check) checks the layout geometry against the foundry rules — minimum widths, spacings, enclosures and densities — so that the chip can actually be manufactured. LVS (layout versus schematic) extracts the transistors and connections from the layout and compares them with the netlist, so that the layout is the intended circuit, with no shorts, opens or missing connections. Both must pass because a DRC-clean layout can still be the wrong circuit, and a correct circuit can

still break manufacturing rules. This design has **0 DRC errors** (Magic, KLayout and routing) and **0 LVS errors**.

Q6. Scaling to 8 bits

Full adders: $N(N-1)$ goes from $4 \times 3 = \mathbf{12}$ to $8 \times 7 = \mathbf{56}$ (4.7×); AND gates from 16 to 64. Critical path: about $2N$ full-adder delays, from **8 to 16**, i.e. roughly double. Using this design's measured 2.156 ns for 8 full-adder delays (about 0.27 ns each), an 8-bit version needs about $16 \times 0.27 = 4.31$ ns of logic plus 0.52 ns of flip-flop overhead, so $T_{\text{min}} \approx$ **4.83 ns** and $F_{\text{max}} \approx$ **207 MHz**. These 8-bit figures are a projection from the 4-bit measurements, not results of an implemented 8-bit design. Area grows about as N^2 (4×) and delay about as N (2×).

7 Version History and Deliverables

Table 7: Git commit history of `github.com/AnumayRai/RTL-to-GDS` (branch `main`) at the time of writing.

Commit	Date	Message
`85d4d1a`	19 Oct 2026, 05:17	RTL and TB at 19/10/2025 05:17 AM
`140fdff`	24 Oct 2026, 03:56	Complete Part B synthesis and gate-level verification
`7469e4d`	24 Oct 2026, 04:18	Part B Change in code : 24/10/2026 : 04:18 AM
`a05b73a`	24 Oct 2026, 20:57	Part C and D not Finalized 24/10/2026 20:57
`957a58a`	25 Oct 2026, 05:49	Complete Project 25/10/2026 5:49
`8386083`	25 Oct 2026, 18:10	Remove PnR run folders and unused adder; add 10 ns STA evidence, decimal waveforms, updated README

Table 8: Deliverables checklist (Section 11 of the assignment).

Part	Deliverable	Location
All	Repository with the required structure; README with both members and every command in order	`README.md`
A	`full_adder.v`, `mult_array.v`, `tb_mult.v`	`rtl/`, `tb/`
A	PASS line, FAIL line, waveform with latency, hand-worked example, block diagram	Sections 2.3–2.6
B	`synth.tcl`, netlist, stat report; Part B table; netlist PASS line	`synth/`; Table 2
C	`sta.tcl`, timing report; Part C table; cells on the critical path	`sta/`; Tables 3, 4
D	`config.json`, `metrics.json`, GDS; Part D table; two layout screenshots	`pnr/`; Table 6; Figs. 4, 5
Report	Answers to questions 1–6	Section 6